



Traceability Model

Overview

This document defines all allowed traceability and structural relations between domain entities, their cardinalities, forbidden relations, circular dependency rules, and deletion cascade policy. The model is deterministic and machine-readable to support schema design, API contracts, and AI-driven updates.

Relation Types

Definition Table

Relation Name	Source Entity	Target Entity	Cardinality (Source → Target)	Description
part-of	Requirement	PBS	1..1	Requirement is scoped to one PBS node (project_id).
part-of	Test Case	PBS	1..1	Test case is scoped to one PBS node (project_id).
part-of	Test Plan	PBS	1..1	Test plan is scoped to one PBS node (project_id).
part-of	Function	PBS	1..1	Function is scoped to one PBS node (project_id).
part-of	PBS	PBS	0..1	PBS node has one

Relation Name	Source Entity	Target Entity	Cardinality (Source → Target)	Description
				parent (hierarchy).
verifies	Test Case	Requirement	1..n	Test case verifies one or more requirements.
derives-from	Requirement	Requirement	0..1	Child requirement derives from one parent.
satisfies	Function	Requirement	1..n	Function satisfies one or more requirements.
executed-in	Test Run	Test Case	1..1	Test run is one execution of one test case.
executes_plan	Test Run	Test Plan	0..1	Test run may belong to one test plan.
contains	Test Plan	Test Case	0..n	Test plan contains zero or more test cases (many-to-many).
parent	Function	Function	0..1	Function may have one parent function.
parent	PBS	PBS	0..1	PBS node may have one parent node.
created_by	All domain entities with created_by_id	User	0..1	Creator (optional).

Relation Name	Source Entity	Target Entity	Cardinality (Source → Target)	Description
updated_by	All domain entities with updated_by_id	User	0..1	Last updater (optional).
executed_by	Test Run	User	0..1	Executor (optional).
has_role	User	Role	0..n	User has zero or more roles (see access-control-model).

Allowed Cardinalities (Summary)

Relation	Allowed Cardinality	Constraint
Requirement → PBS	1..1	Every requirement has exactly one project_id.
Test Case → PBS	1..1	Every test case has exactly one project_id.
Test Plan → PBS	1..1	Every test plan has exactly one project_id.
Function → PBS	1..1	Every function has exactly one project_id.
PBS → PBS (parent)	0..1	Every node has at most one parent.
Test Case → Requirement (verifies)	1..n	Every test case verifies at least one requirement.

Relation	Allowed Cardinality	Constraint
Requirement → Requirement (derives-from)	0..1	Every requirement has at most one parent requirement.
Function → Requirement (satisfies)	1..n	Every function satisfies at least one requirement.
Test Run → Test Case	1..1	Every test run executes exactly one test case.
Test Run → Test Plan	0..1	Every test run has at most one test plan.
Test Plan ↔ Test Case (contains)	0..n each	Many-to-many.
Function → Function (parent)	0..1	Every function has at most one parent function.

Forbidden Relations

The following relations are NOT allowed:

Source	Target	Reason
Requirement	Test Case	Direction is Test Case → Requirement (verifies). Requirement does not "point to" test case.
Requirement	Function	Direction is Function → Requirement (satisfies).
Test Run	Requirement	Traceability is Test Run → Test Case → Requirement. No direct Test Run → Requirement.
PBS	Requirement / Test Case /	Direction is entity → PBS (part-of). PBS does not own a list of entities as a relation table; ownership is

Source	Target	Reason
	Function / Test Plan	via project_id on entity.
Test Case	Test Run	Direction is Test Run → Test Case (executed-in).
User	Requirement / Test Case / etc.	Attribution is entity → User (created_by, updated_by).

Circular Dependency Rules

Entity Pair	Rule
Requirement ↔ Requirement	derives-from SHALL form a directed acyclic graph (DAG). No cycle: R1 → R2 → R1 is forbidden.
PBS ↔ PBS	parent_id SHALL form a tree. No cycle: N1 → N2 → N1 forbidden.
Function ↔ Function	parent_function_id SHALL form a tree. No cycle.
Test Case ↔ Requirement	verifies is many-to-many with no hierarchy; no cycle possible.
Function ↔ Requirement	satisfies is many-to-many; no cycle possible.

Validation: On create/update of derives-from, parent_id, or parent_function_id, the system SHALL check that the graph remains acyclic (and tree for PBS/Function).

Deletion Cascade Policy

Entity Deleted	Relation	Cascade Behavior
PBS	part-of (Requirements, Test Cases, etc.)	Do NOT cascade delete. Block PBS delete if any Requirement/Test Case/Test Plan/Function has project_id = this node; or reassign project_id to another node first.
PBS	children	Block delete if node has children; or move/delete children first per policy.
Requirement	derives-from (children)	On delete of parent: reassign child.derived_by to null or block delete. No cascade delete of children.
Requirement	verifies (Test Case links)	On delete of Requirement: remove all verifies links (junction rows). Test Case remains.
Test Case	verifies	On delete of Test Case: remove all verifies links. Requirement remains.
Test Case	executed-in (Test Runs)	Do NOT cascade delete Test Runs. Retain Test Run with nullable test_case_id or block Test Case delete while runs exist (configurable).
Test Plan	contains	On delete of Test Plan: remove all contains links. Test Cases remain.
Test Run	—	No outgoing relations that cascade. Deletion of Test Case/Test Plan does not delete Test Run (see above).
Function	satisfies	On delete of Function: remove all satisfies links. Requirements remain.
Function	parent (children)	On delete of parent: set child.parent_function_id = null or block delete.
User	created_by / updated_by /	On user delete: set FK to null or retain id with "deleted user" label. Do not delete domain entities.

Entity Deleted	Relation	Cascade Behavior
	executed_by	

Summary: **No cascade delete of domain entities** (Requirement, Test Case, Test Plan, Test Run, Function, PBS) when a related entity is deleted. Only **link rows** (verifies, satisfies, contains, derives-from) are removed when one side is deleted, unless policy explicitly requires blocking delete.

Junction Tables (Many-to-Many)

Junction Table	Left Entity	Right Entity	Relation	Notes
requirement_test_case (verifies)	Test Case	Requirement	Test Case 1..n Requirement	Store test_case_id, requirement_id. Unique (test_case_id, requirement_id).
test_plan_test_case (contains)	Test Plan	Test Case	Test Plan 0..n Test Case	Store test_plan_id, test_case_id. Unique (test_plan_id, test_case_id). Optional order_index.
function_requirement (satisfies)	Function	Requirement	Function 1..n Requirement	Store function_id, requirement_id. Unique

Junction Table	Left Entity	Right Entity	Relation	Notes
				(function_id, requirement_id).

Change Impact Notes

- `[ADD_RELATION]` Adding a new relation type: new FK or junction table; API and UI for link management; update this document.
- `[REMOVE_RELATION]` Removing a relation: breaking; migration to drop FK/junction; remove from API and traceability UI.
- `[CHANGE_CARDINALITY]` Changing cardinality: migration (e.g. 0..1 → 1..1 adds NOT NULL constraint); validation and cascade policy review.
- `[CHANGE_CASCADE]` Changing cascade policy: update delete handlers and possibly migration for historical data.